

StatefulSet (HA)

What is stateful application?

- A stateful application is a program that saves client data from the activities of one session for use in the next session. The data that is saved is called the application state e.g., banking app.
- Stateful applications requires that data that is used or generated by application is persistent, retained, backed up and accessible outside of the particular host that run the application.
- Stateful applications may expect to have static host name of its replicas.
- Stateful applications may expect to start from a start point not as a whole.

What is StatefulSet in K8s?

- about the ordering and uniqueness of these pods.
- keep record of its state.
- Unlike a Deployment, a StatefulSet maintains a sticky Identity for each of their Pods.
-

Ordered, graceful deployment and scaling

The pods are deployed in order (It creates a first pod and the waits till it up and running then creates second pod, continue the same order until last pod creation). It is the same with scale up or scale down again (when we scale down it brings down the last indexed pod first and goes down accordingly).

Stable, persistent storage

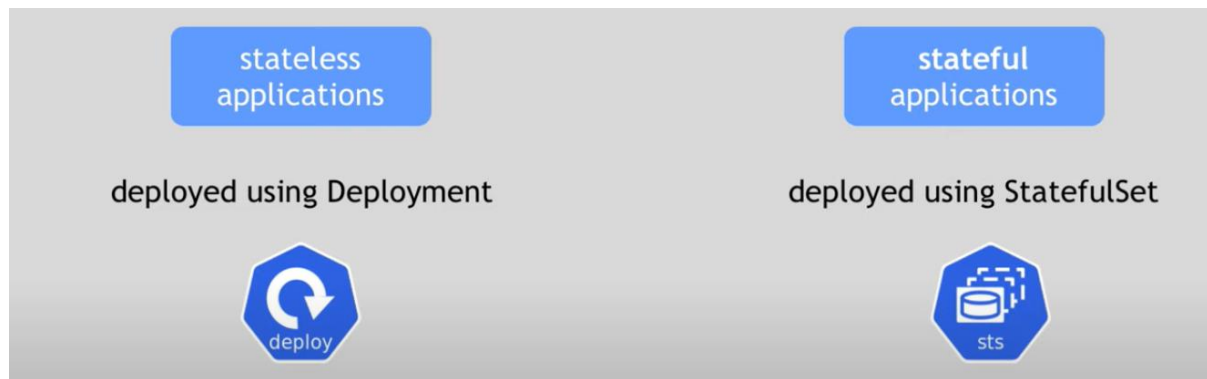
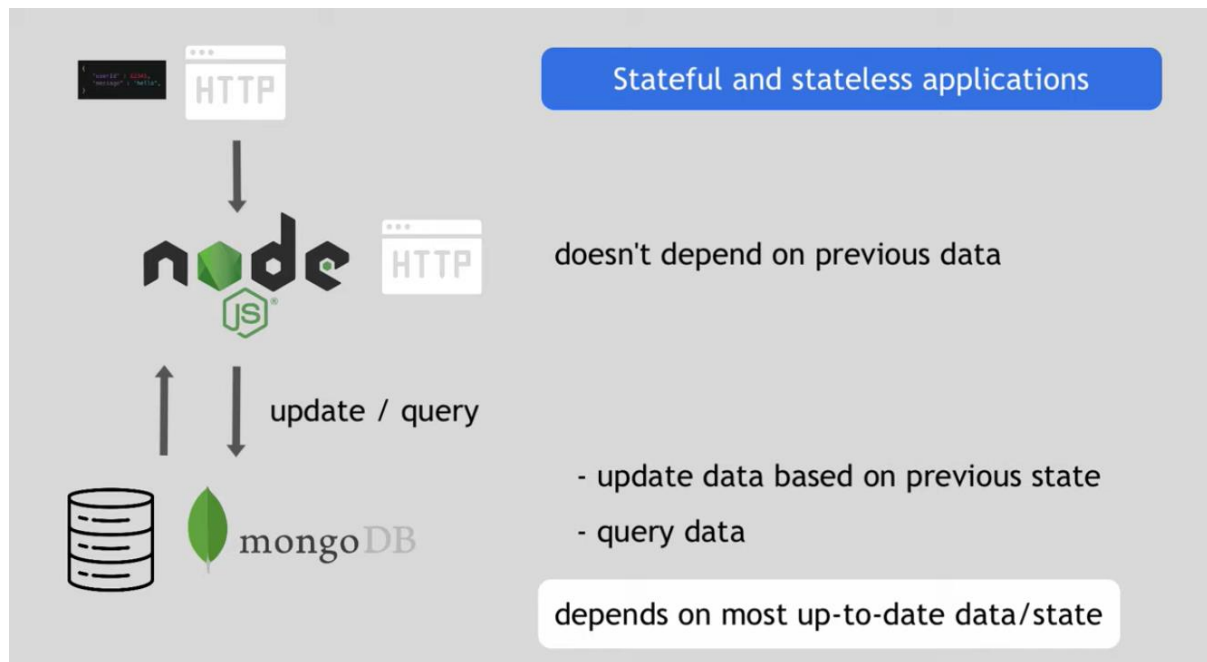
Each pod in StatefulSet will have its own PV created, the data replication needs to be managed by application or external way between all pods writing data to their own persistent disks.

When StatefulSet pod is deleted the PV/PVC doesn't get deleted. It has to be taken care manually.

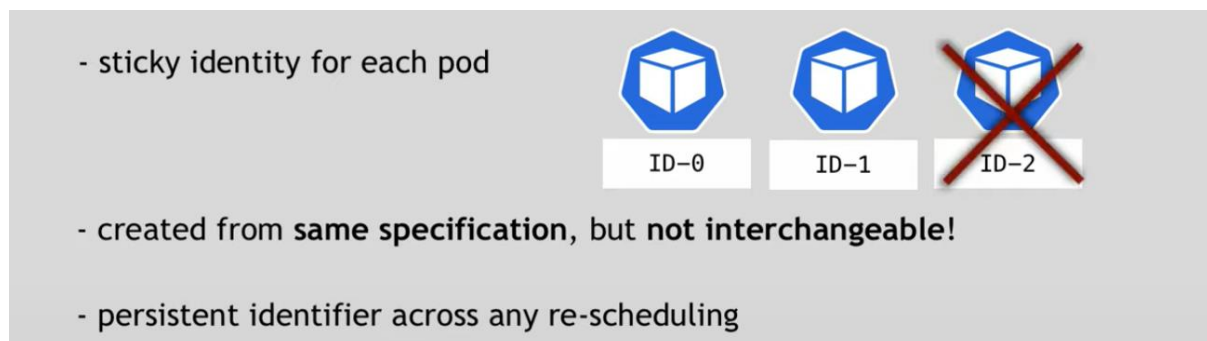
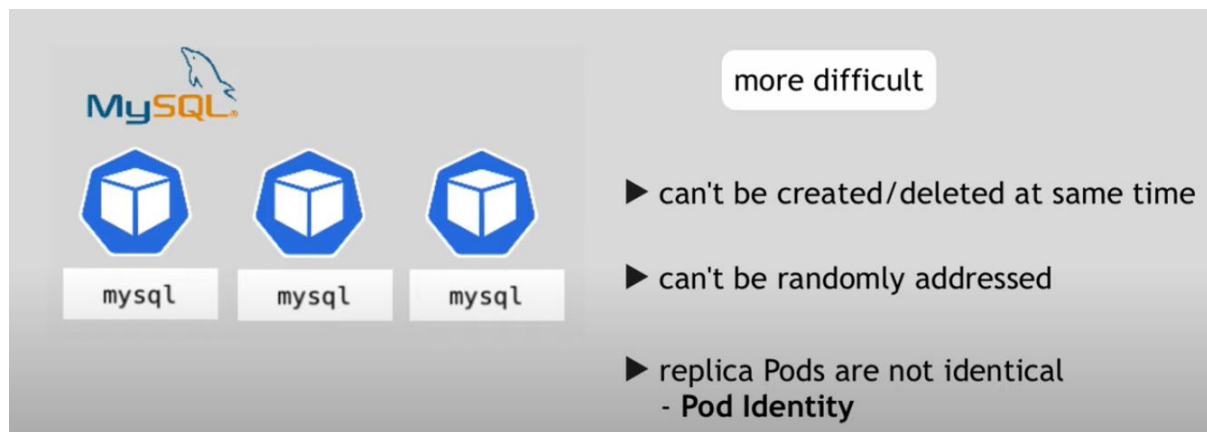
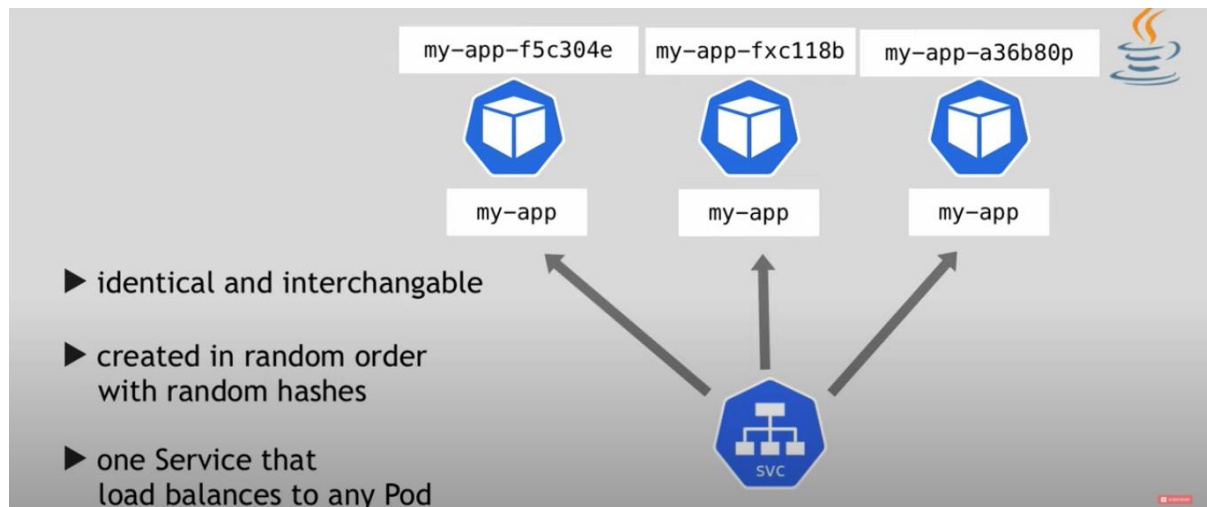
Headless Service

A StatefulSet always requires a headless service. There is no cluster IP, instead there is service name, which we define in StatefulSet.

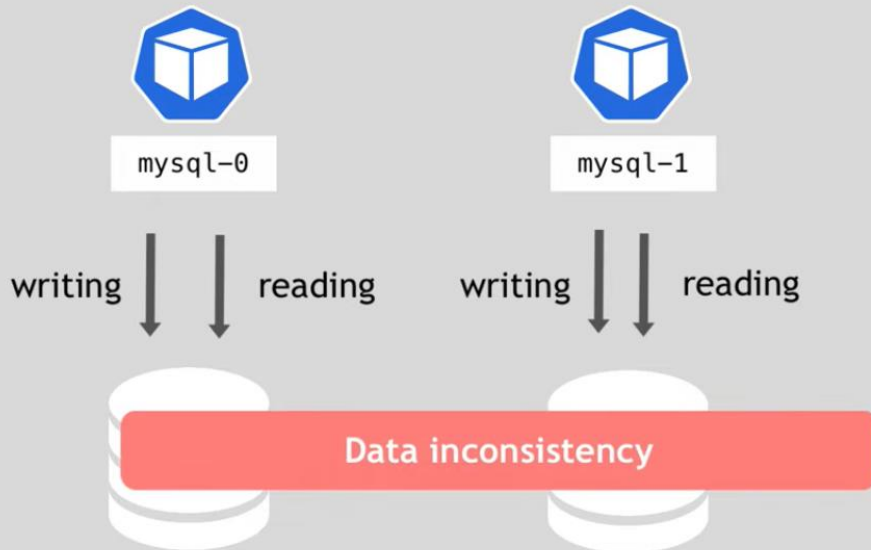
UC: deploy database server, our objective is to provide HA solution.



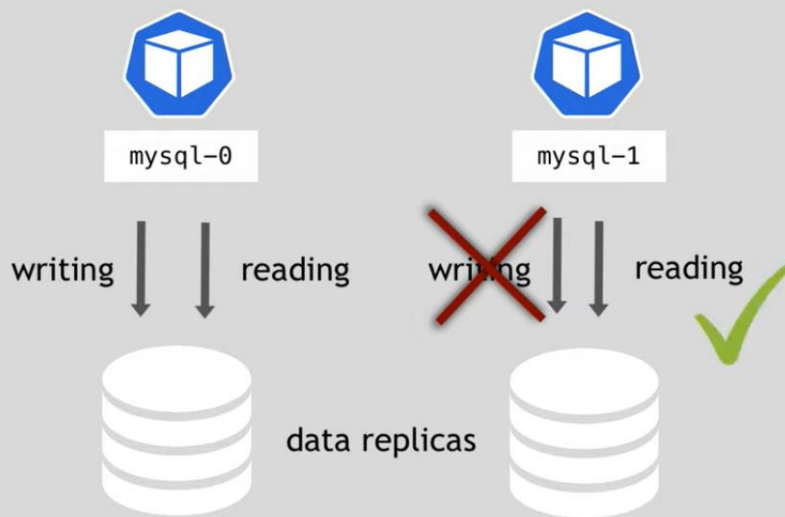
- replicating stateful applications is more difficult
- other requirements

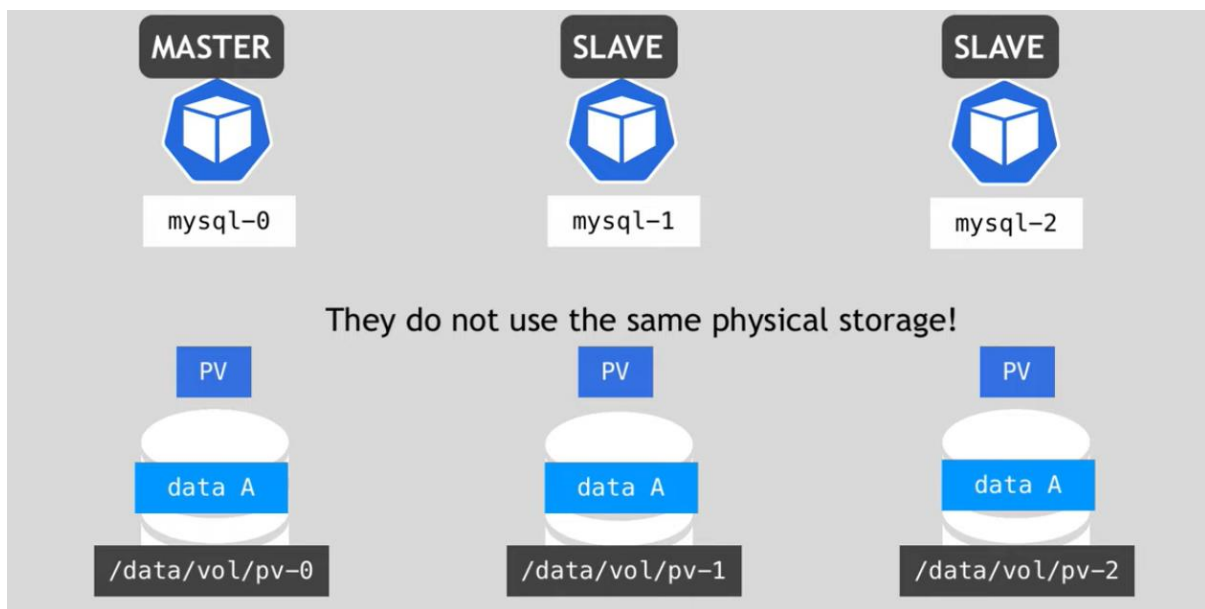
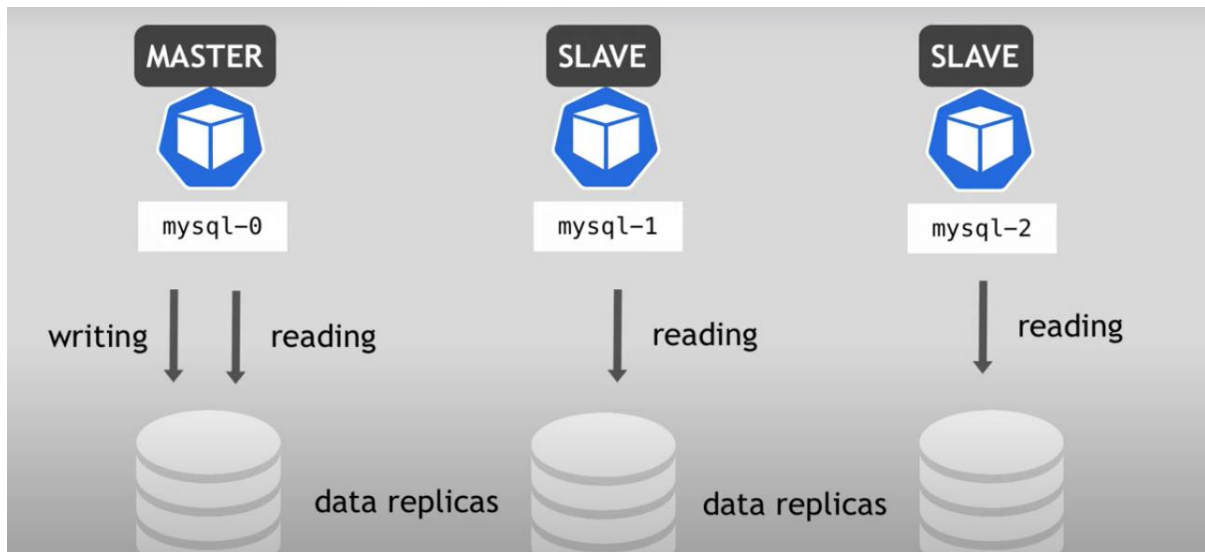


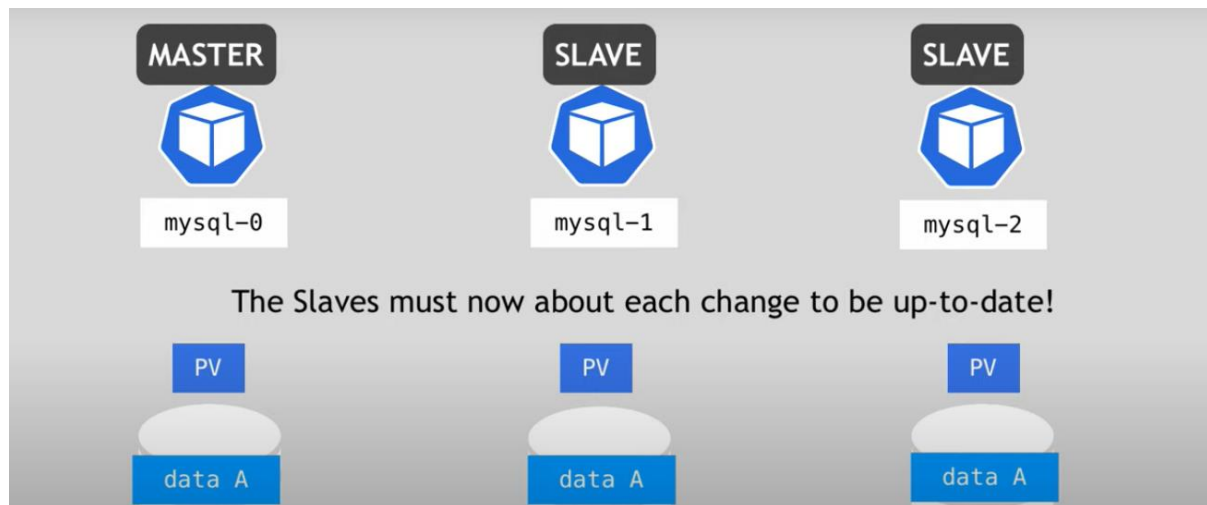
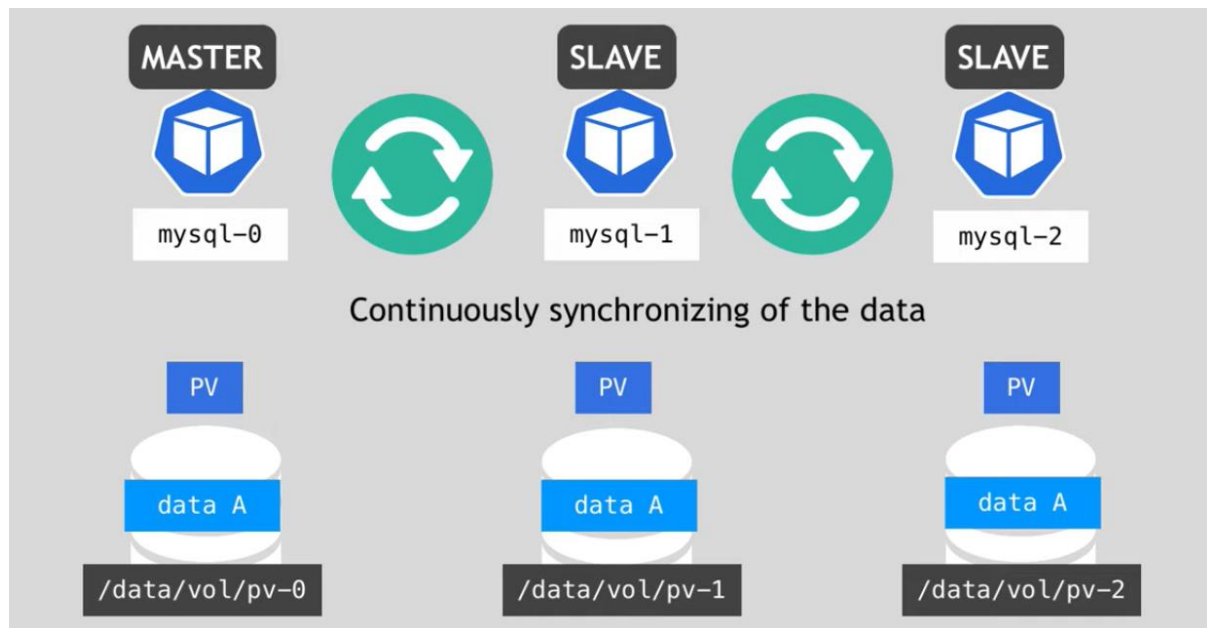
Scaling database applications

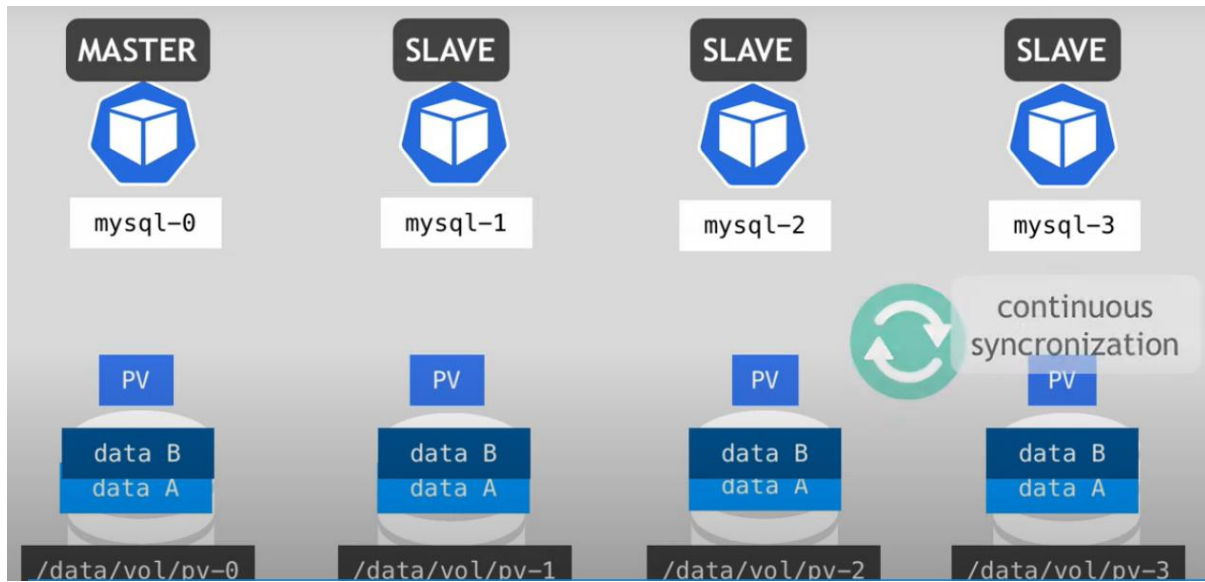
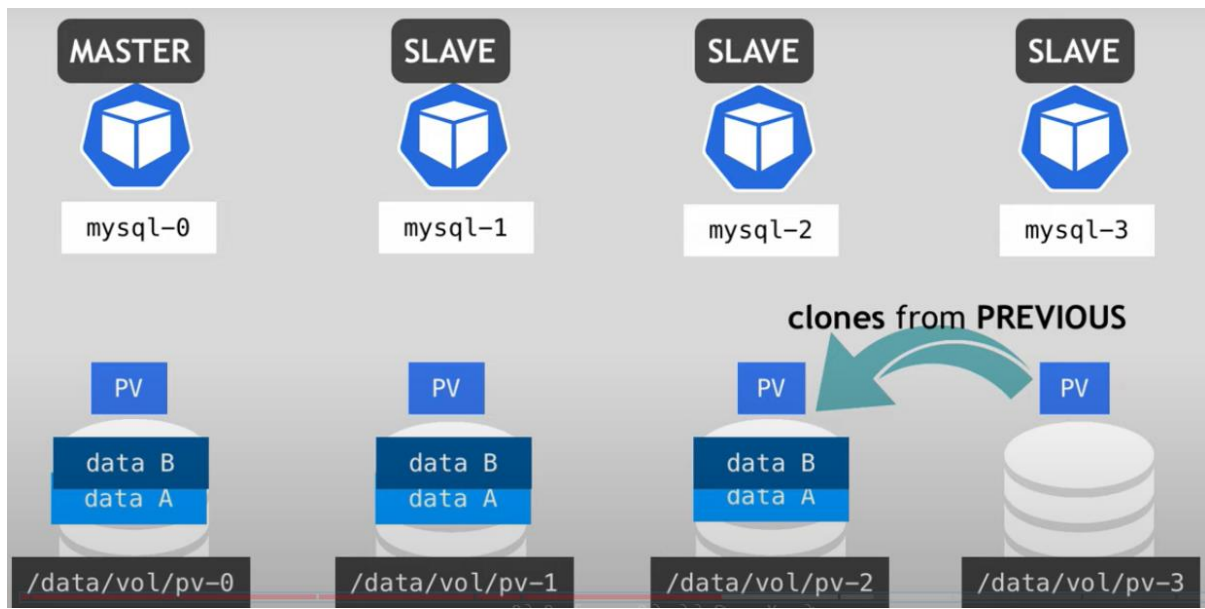


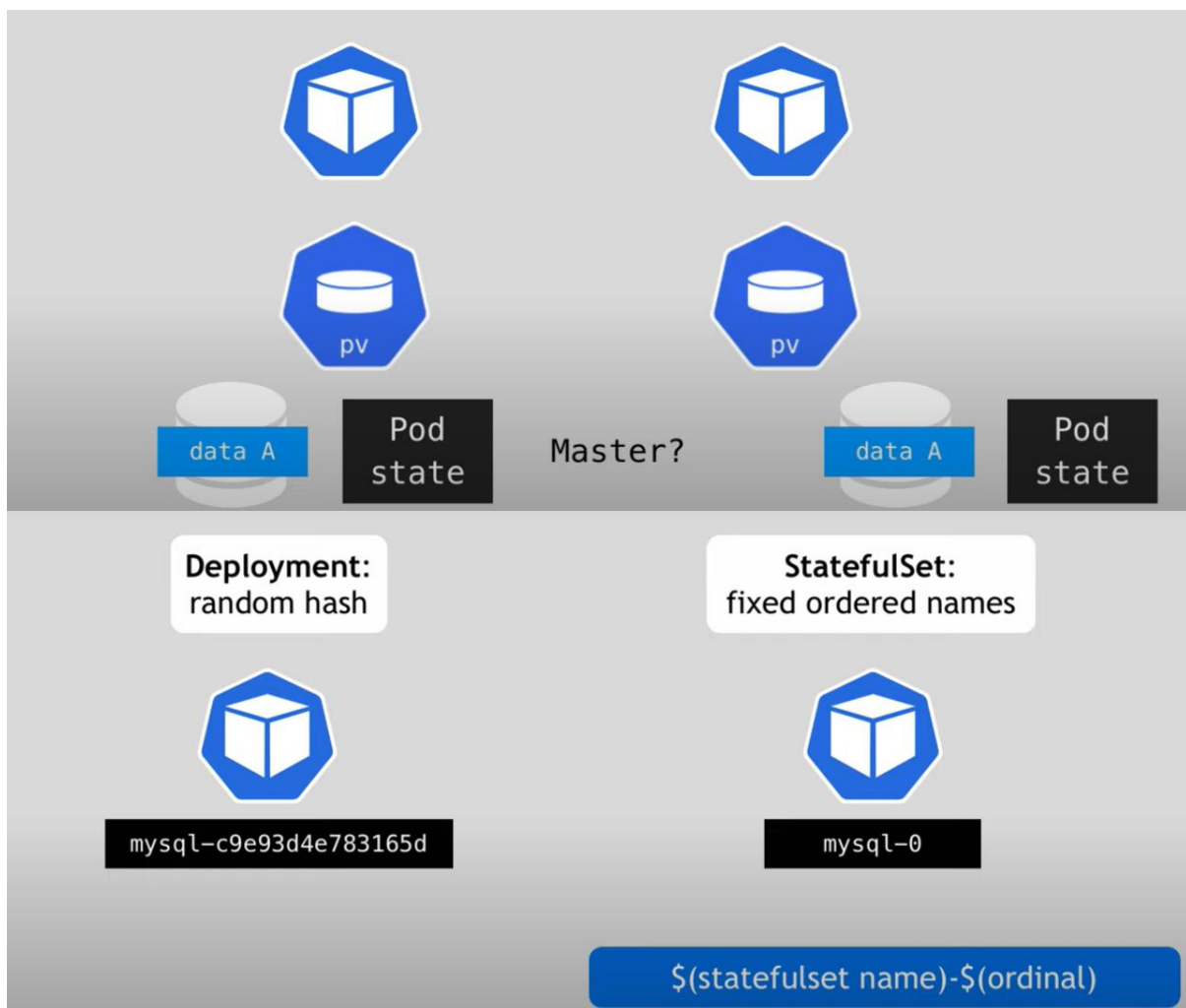
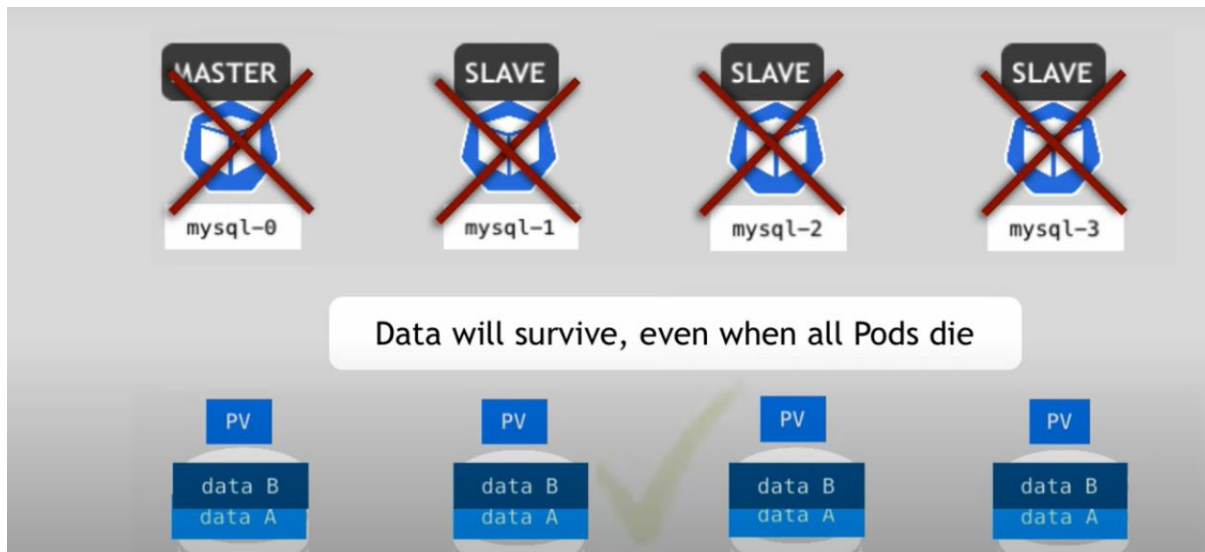
Scaling database applications











StatefulSet with 3 replica



mysql-0

MASTER



mysql-1

SLAVE



mysql-2

SLAVE

StatefulSet with 3 replica



mysql-0

Next Pod is only created, if previous
is up and running!

Delete StatefulSet or scale down to 1 replica



mysql-0

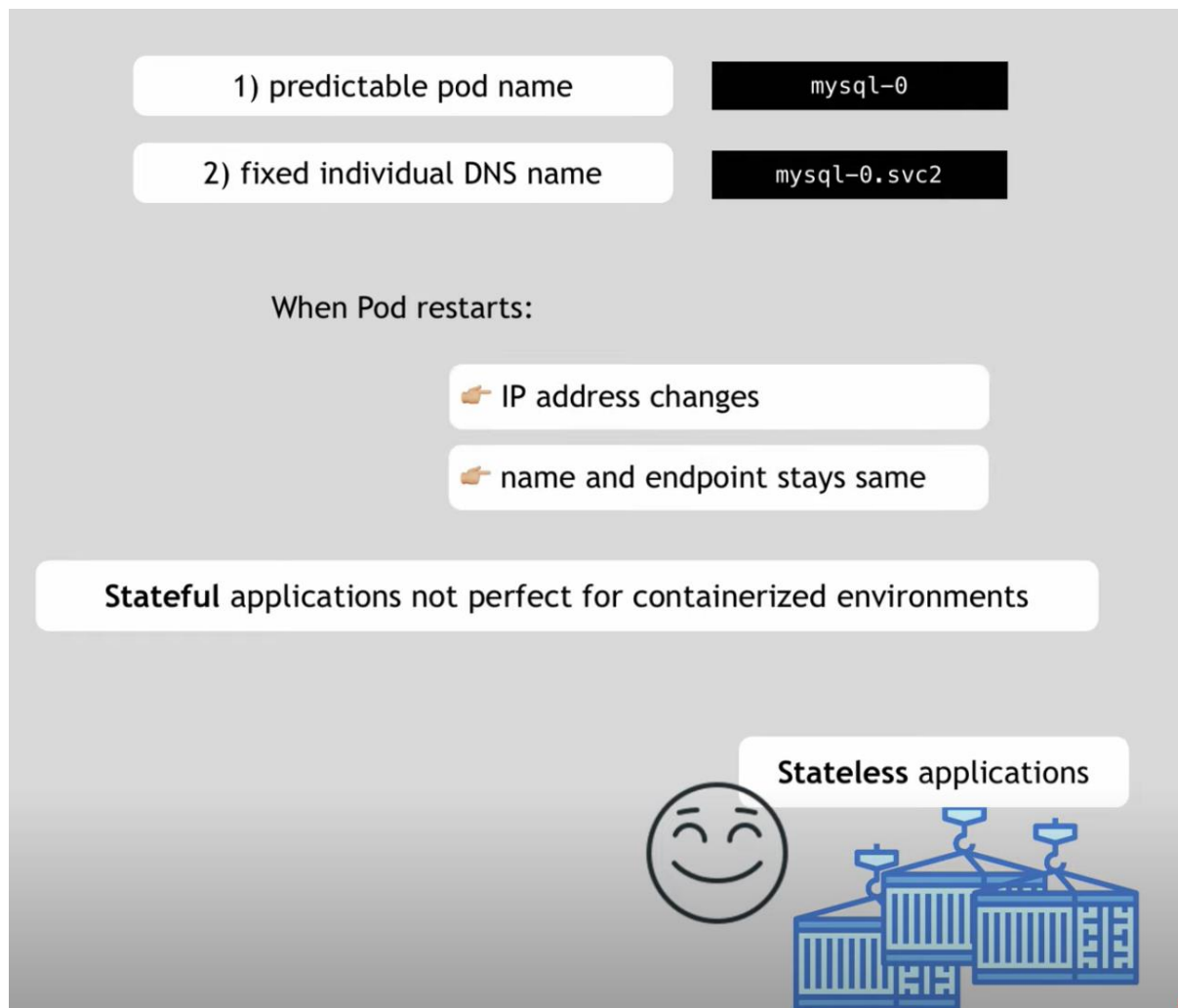


mysql-1



mysql-2

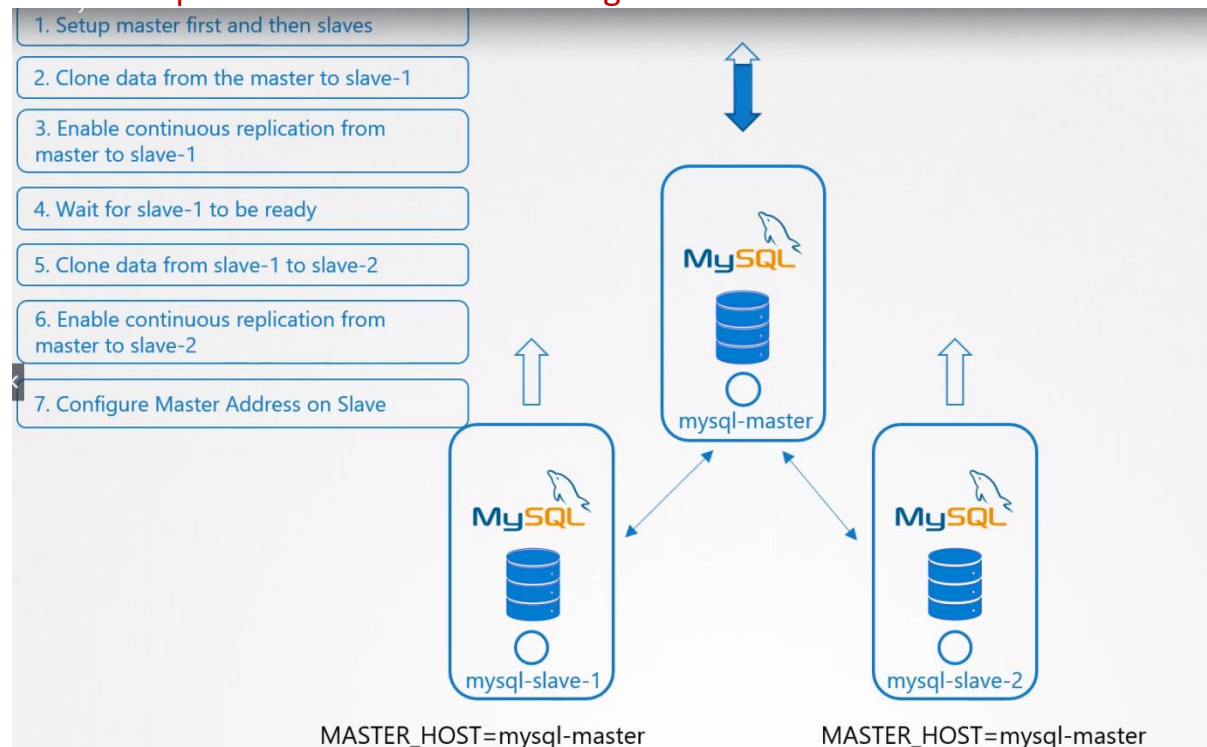
Deletion in reverse order, starting
from the last one



Topology1: Single master and multiple slaves

Write on master, read from either master or from any of the slave. So, the master should be deployed first before any of the slave. Once slave is deployed perform the initial clone of the database from the master server to the first slave, after initializing enable continuous replication from master to the slave. Then we setup the second slave, since we have a copy of master in slave1, replicate from slave1 to slave2.

How we replicate the data from the original db server to the new servers?



Problem Statement in world of K8s:

With the deployment all the part of the Deployment comes up at the same time. As per our use case we want Master comes first and after that node1 and after that node2, but with Deployments we can't guarantee the order, all pods part of the deployment comes up at same time, so first step can't be implemented with the deployment. We can't rely on IP address of the Pod as we know that IP address dynamically assigned to Pod and they get change when they recreated. With deployment Pods come with random name.

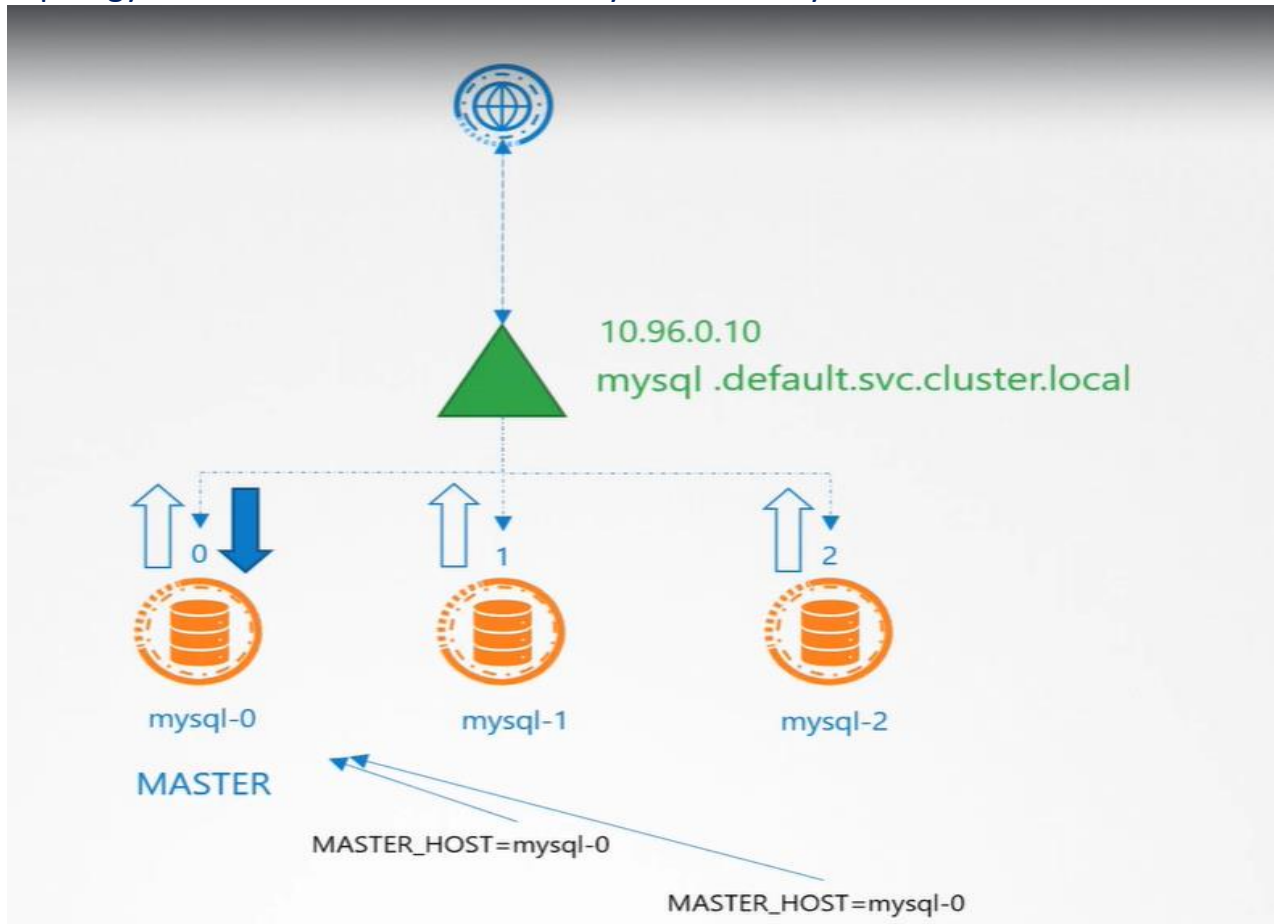
Stateful Sets

Similar to deployment as they create a Pods based on the template, can scale up, scale down can perform rolling update, with some difference. In Stateful set pods are created in sequential order after the first pod is deployed it must be running and ready state before second pod is deployed that will ensure that master pod deployed first and then slave1 and after that slave2 will be deployed.

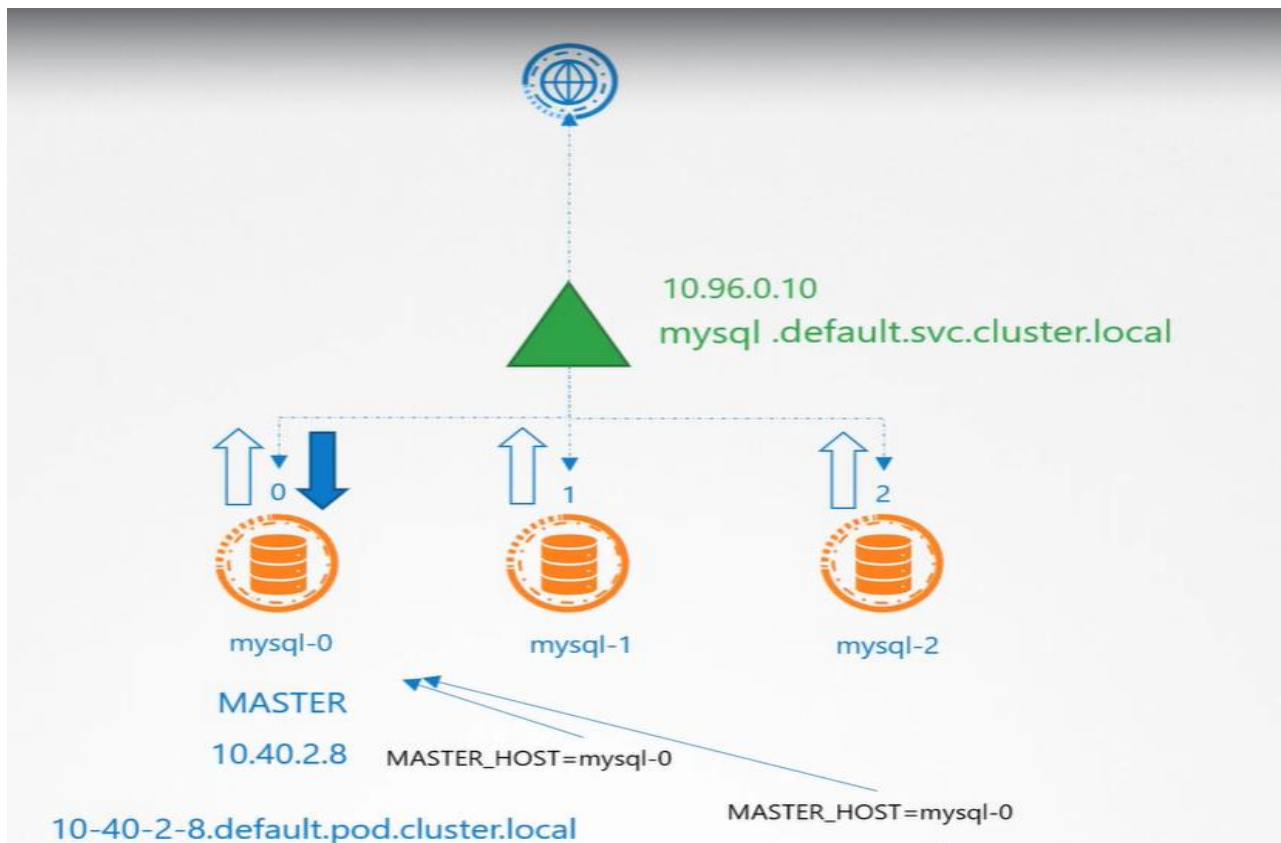
Stateful set assign a unique index id to each Pod, a number 0 assigned to first pod and incremented by 1, each pod is assigned unique name (sticky session), no more random name. Stateful set maintains a sticky Identity for each of their Pod, Pod maintains its Identity, master is always master.

Headless Services

As we aware application in K8s access through the service, service also acts a Load balancer. Service can be accessed through the DNS name, Since Its Master-Slave topology so read can be served either by master or by slave.



UC: write only to the master only, **how to write only to Master?** Can configure the IP of master in Service but IP are dynamically can be change, can be access through the Pod's DNS address but Pod's DNS addresses created from its IP address.



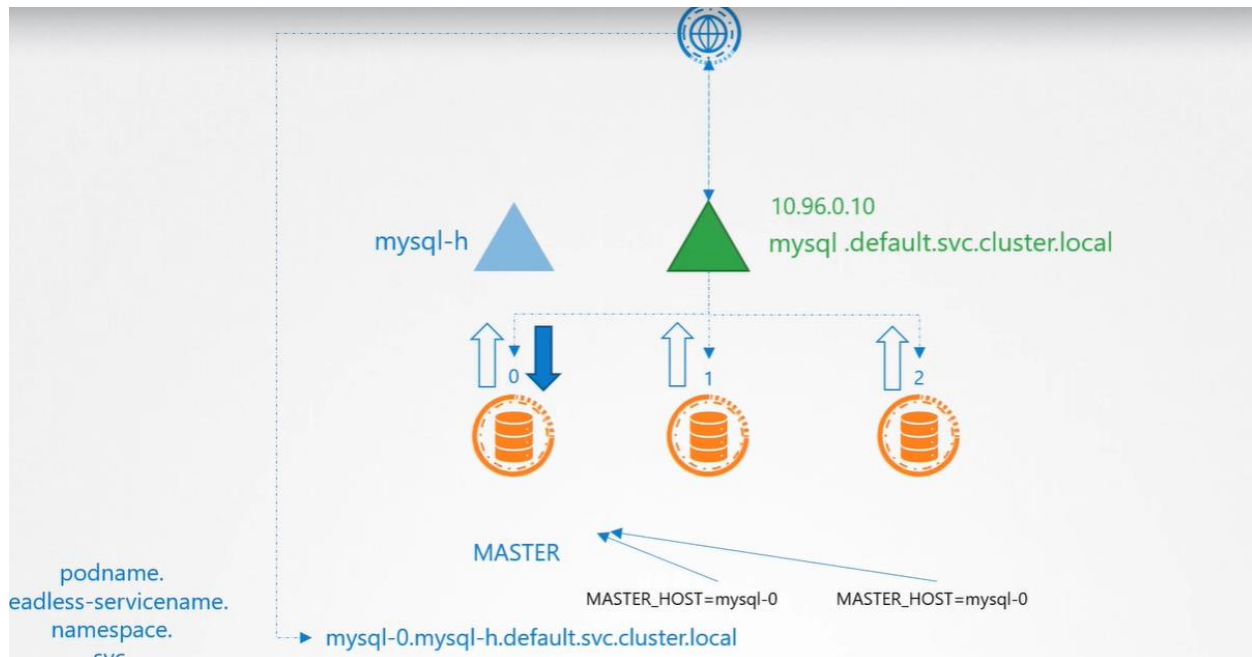
Our requirement is service **which don't Load Balance** the request but **give us the DNS entry to reach each Pod**.

Solution: Headless Service

Headless service is created like a normal service, **it doesn't have its own IP like ClusterIP for a normal service (ClusterIP=None)**, and it does not perform any load balancing. All it does it creates a DNS entry for each Pod using the Pod name and sub domain. So, when you create a Headless service (mysql-h), each pod gets a DNS record created in the form

<Podname>. <headless-svcname>. <namespace>. <subdomain>. <defaultdomain>

mysql-0.mysql-h.default.svc.cluster.local

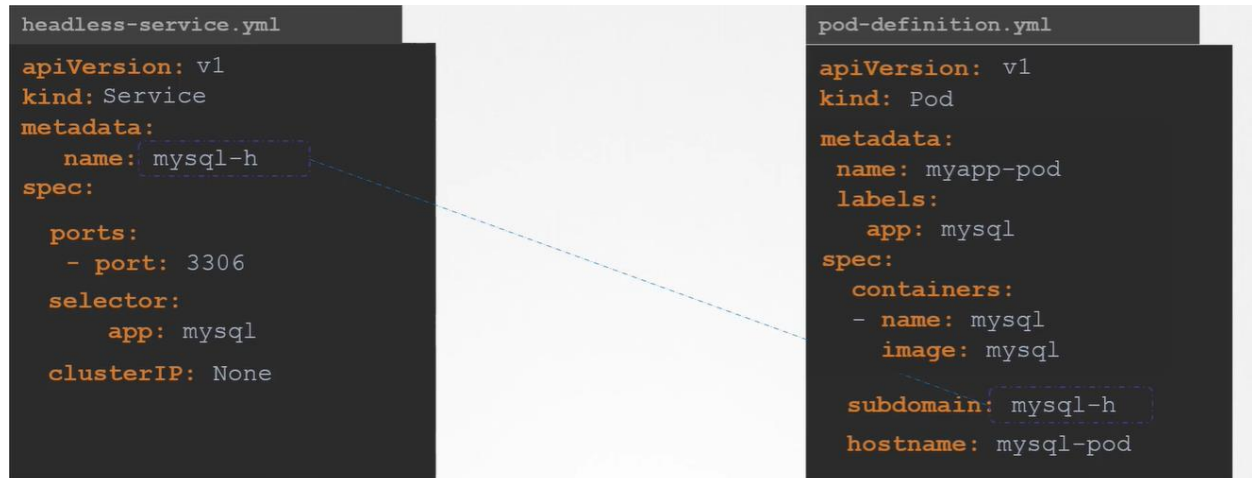


The Web application can now point to the DNS entry for the master my-sql server at

mysql-0.mysql-h.default.svc.cluster.local

That DNS entry always point to the master Pod in my-sql deployment.

UC:



Creating the service using this definition file creates a Headless Service. When **Headless service is created a DNS, entries are created for the Pods only if two conditions (subdomain and hostname) are met while creating the Pod.**

Limitations in deployment:

If we define the subdomain or hostname in deployment then it will assign the same hostname and subdomain for all the pods, because deployment simply duplicates all the properties, this resulting creating an entry for all pods with same name.

headless-service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-h
spec:
  ports:
    - port: 3306
  selector:
    app: mysql
  clusterIP: None
```

5 mysql-pod.mysql-h.default.svc.cluster.local

deployment-definition.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql-deployment
  labels:
    app: mysql
spec:
  replicas: 3
  matchLabels:
    app: mysql
  template:
    metadata:
      name: myapp-pod
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql
          subdomain: mysql-h
```

mysql-pod.mysql-h.default.svc. cluster. local

mysql-pod.mysql-h.default.svc. cluster. local

mysql-pod.mysql-h.default.svc. cluster. local

Solution: Just replace kind Deployment-> StatefulSet

Storage in StatefulSet

By default, same volume associated with all the Pods

pv-definition.yaml

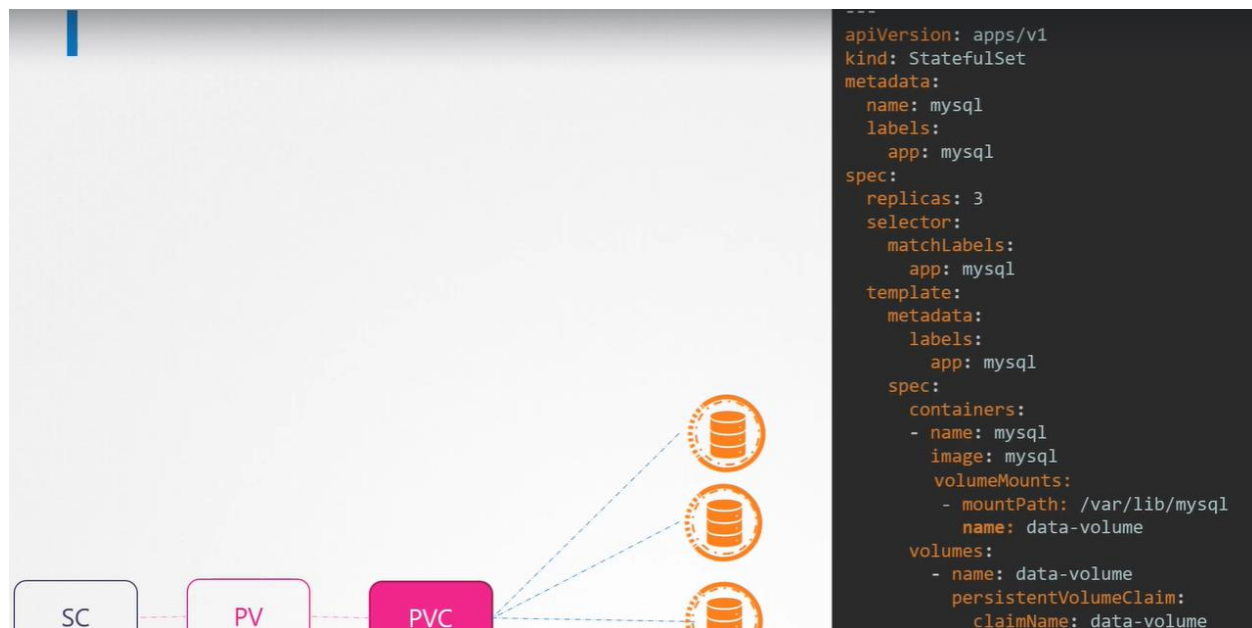
```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

pvc-definition.yaml

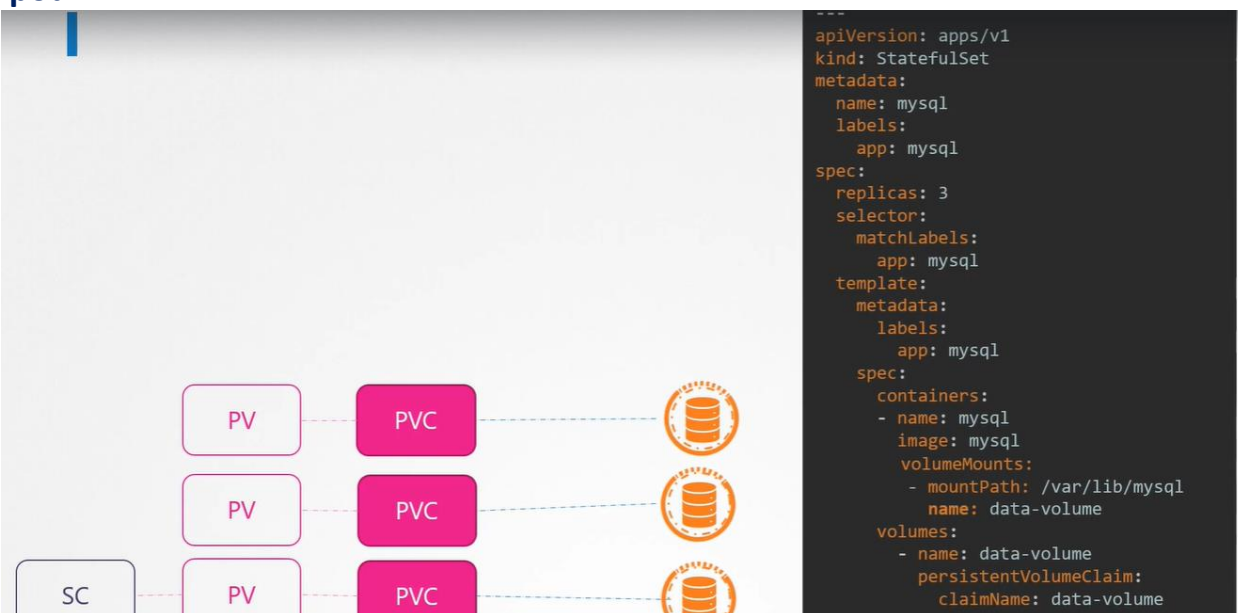
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-volume
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: google-storage
  resources:
    requests:
      storage: 500Mi
```

pod-definition.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: mysql
spec:
  containers:
    - image: mysql
      name: mysql
      volumeMounts:
        - mountPath: /var/lib/mysql
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: data-volume
```

Every Stateful application associated with volume, separate storage for each pod



How to assign separate storage for each Pod?

volumeClaimTemplates

pvc-definition.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
```

```
replicas: 3
selector:
  matchLabels:
    app: mysql
template:
  metadata:
    labels:
      app: mysql
  spec:
    containers:
      - name: mysql
        image: mysql
        volumeMounts:
          - mountPath: /var/lib/mysql
            name: data-volume
    volumeClaimTemplates:
      - metadata:
          name: data-volume
        spec:
          accessModes:
            - ReadWriteOnce
          storageClassName: google-storage
          resources:
            requests:
```

Replicating stateful apps

► it's complex

► Kubernetes helps you

► You still need to do a lot:

- Configuring the cloning and data synchronization
- Make remote storage available
- Managing and back-up

